JavaFX常用控件之

使用系统对话框

北京理工大学计算机学院
金旭亮

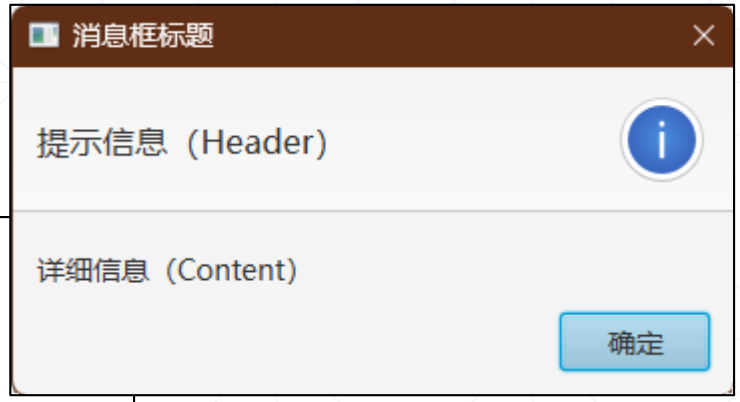
消息框与输入框示例

UseDialog.java

消息框

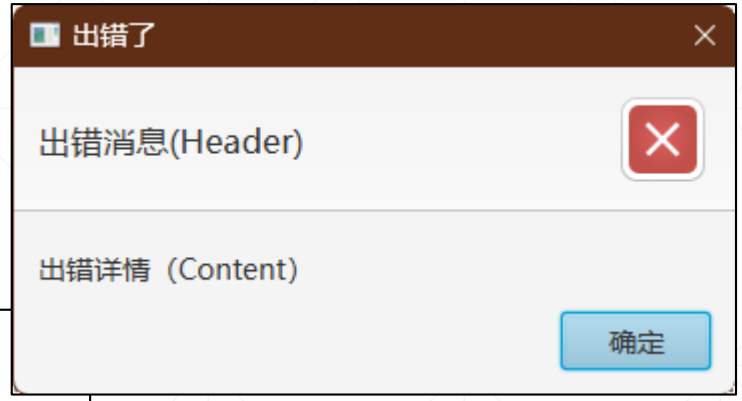
单纯用于显示一条消息。

```
Button btnInfo = new Button( text: "模式消息框");
btnInfo.setOnAction(e -> {
    //设定Alert的类型
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("消息框标题");
    alert.setHeaderText("提示信息（Header）");
    alert.setContentText("详细信息（Content）");
    //显示Alert
    alert.showAndWait();
});
```



显示警告或出错信息

```
Button btnError = new Button( text: "警告或出错提示");
btnError.setOnAction(e -> {
    //可选类型: AlertType.ERROR和AlertType.WARNING
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("出错了");
    alert.setHeaderText("出错消息(Header)");
    alert.setContentText("出错详情 (Content) ");
    alert.showAndWait();
});
```

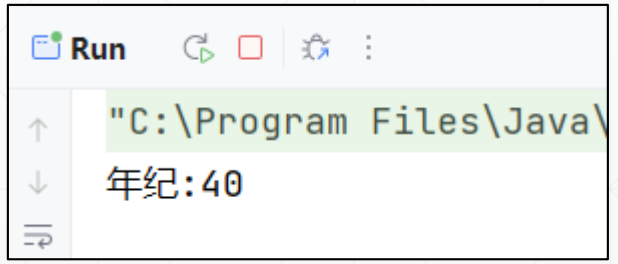


用户选择框

```
Button btnConfirm = new Button( text: "用户选择框");
btnConfirm.setOnAction(e -> {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("请确定");
    alert.setHeaderText("真的需要进行这个操作? ");
    alert.setContentText("此操作将永久删除记录，并且无法恢复");
    Optional<ButtonType> result = alert.showAndWait();
    result.filter(btn -> btn == ButtonType.OK)
            .ifPresent(btn -> {
                System.out.println("用户选择了确定");
            });
    result.filter(btn -> btn == ButtonType.CANCEL)
            .ifPresent(btn -> {
                System.out.println("用户取消了操作");
            });
});
```

用户输入框

```
Button btnTextInputDialog = new Button( text: "用户输入框");
btnTextInputDialog.setOnAction(e -> {
    TextInputDialog dialog = new TextInputDialog();
    dialog.setTitle("请输入");
    dialog.setContentText("您的年纪是多少? ");
    Optional<String> result = dialog.showAndWait();
    if (result.isPresent()) {
        System.out.println("年纪:" + result.get());
    } else {
        System.out.println("用户没有输入");
    }
});
```



下拉列表选择框

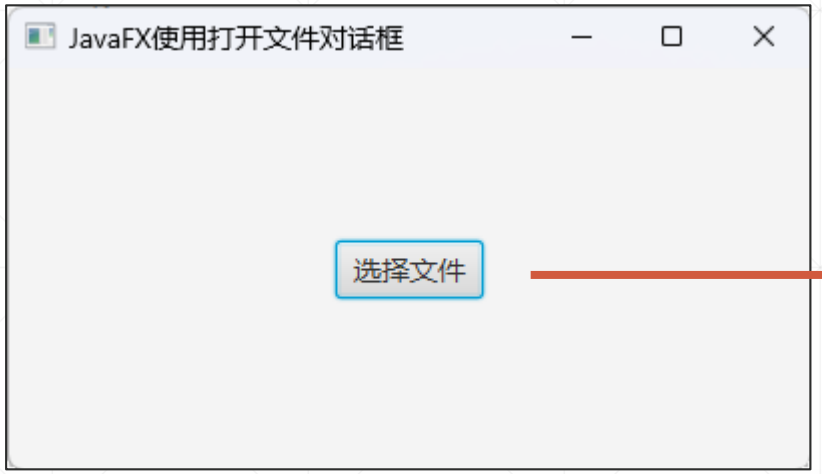

```
Button btnChoiceDialog = new Button( text: "用户选择框");
btnChoiceDialog.setOnAction(e -> {
    ChoiceDialog<String> dialog = new ChoiceDialog<>();
    dialog.setTitle("使用ChoiceDialog");
    dialog.setContentText("请从下拉列表中选中一项");
    dialog.getItems().addAll( ...es: "One", "Two", "Three");
    dialog.setSelectedItem("请选择...");
    dialog.showAndWait().ifPresent(result -> {
        System.out.println("用户选择了: " + result);
    });
});
```

ChoiceDialog<T>的泛型参数<T>，代表了items集合中元素的类型，在运行时，显示在下拉列表中的文本，是这一集合中元素对象toString()方法的返回值，因此，我们可以传给此选择框一个自定义数据对象的集合，只需要重写这个数据类型的toString()方法即可。

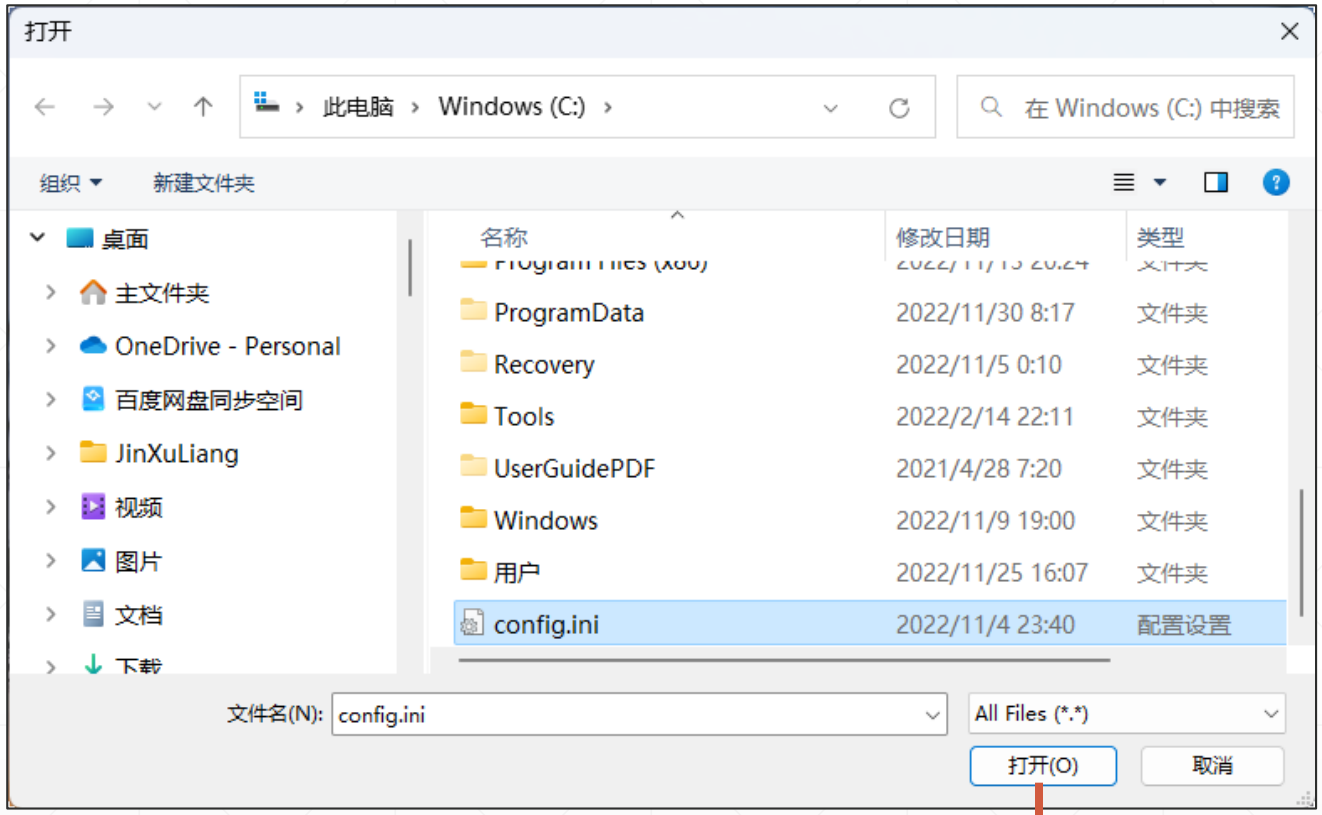
文件与文件夹对话框

JavaFX虽然提供了“自己的”文件（文件夹）打开和保存组件，但与Swing所提的组件不一样，它并没有自己绘制界面，而是直接复用了底层操作系统所提供的对应组件。

打开文件对话框



UseFileChooser.java

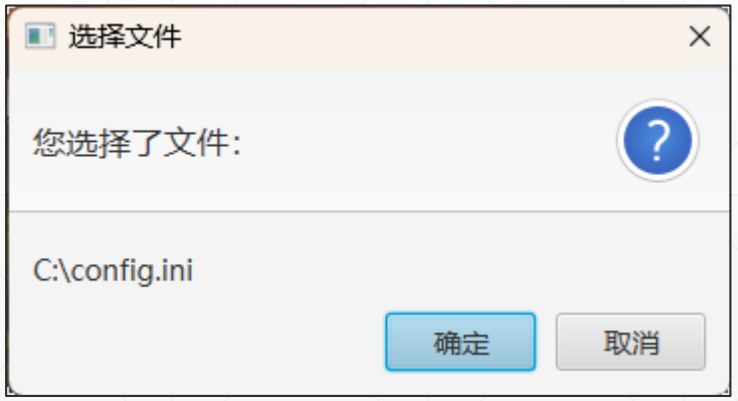


JavaFX提供了一个FileChooser组件，调用操作系统所提供的相应组件，供用户选择要处理的文件。

使用FileChooser选择文件

```
var fileChooser = new FileChooser();
//指定初始显示的文件夹
fileChooser.setInitialDirectory(new File( pathname: "c:"));
//设定筛选过滤条件
fileChooser.getExtensionFilters().addAll(
        new FileChooser.ExtensionFilter( description: "Text Files", ...extensions: "*.txt")
        , new FileChooser.ExtensionFilter( description: "HTML Files", ...extensions: "*.htm")
        , new FileChooser.ExtensionFilter( description: "All Files", ...extensions: "*.*")
);
```

初始设置



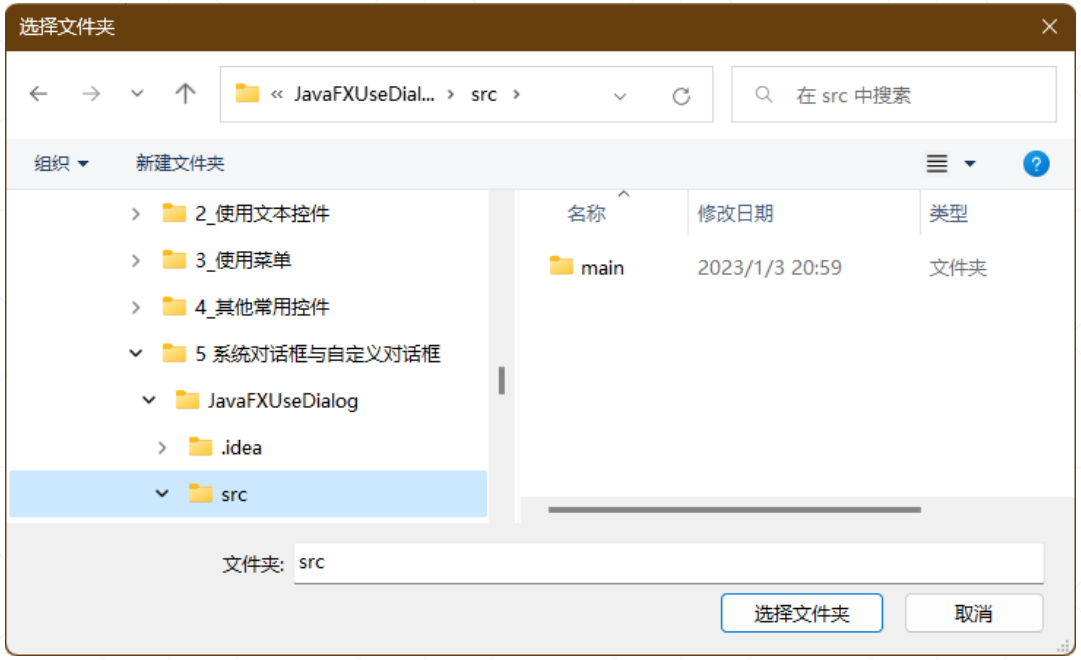

```
Button button = new Button( text: "选择文件");
button.setOnAction(e -> {
    //显示"打开文件"对话框
    File selectedFile = fileChooser.showOpenDialog(primaryStage);
    if (selectedFile != null) {
        var alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle("选择文件");
        alert.setHeaderText("您选择了文件: ");
        alert.setContentText(selectedFile.toString());
        alert.show();
    }

});
```

打开选择多个文件对话框

- FileChooser提供了一个showOpenMultipleDialog()方法用于一次选择多个文件，此方法的返回值为List<File>，可以写代码遍历此集合，取出用户选择的多个文件。
- 如果用户没有选择文件，此方法返回null。

如果要打开“保存为”对话框，可以调用showSaveDialog()方法实现。

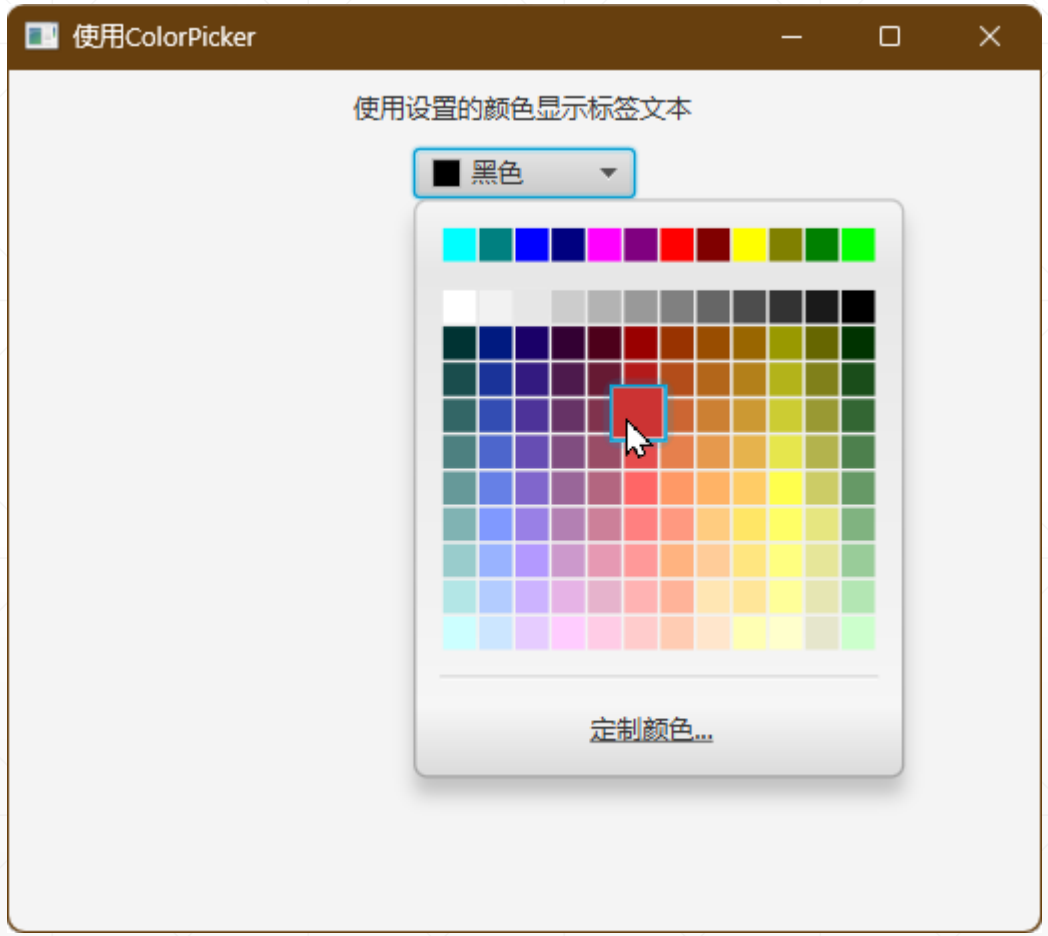
选择文件夹对话框



UseDirectoryChooser.java

```
DirectoryChooser directoryChooser = new DirectoryChooser();
directoryChooser.setInitialDirectory(new File( pathname: "src"));

Button button = new Button( text: "Select Directory");
button.setOnAction(e -> {
    File selectedDirectory = directoryChooser.showDialog(primaryStage);
    if (selectedDirectory != null)
        System.out.println(selectedDirectory.getAbsolutePath());
});
```

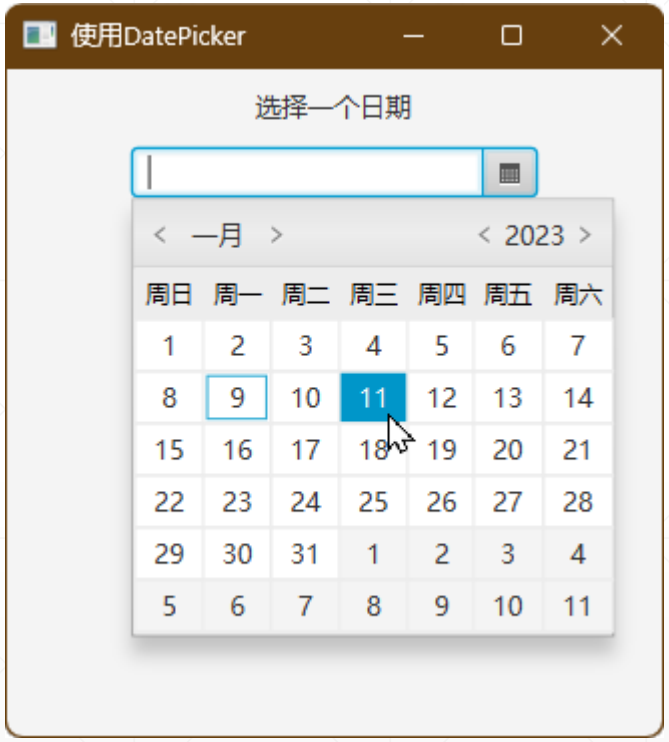
使用ColorPicker选择颜色

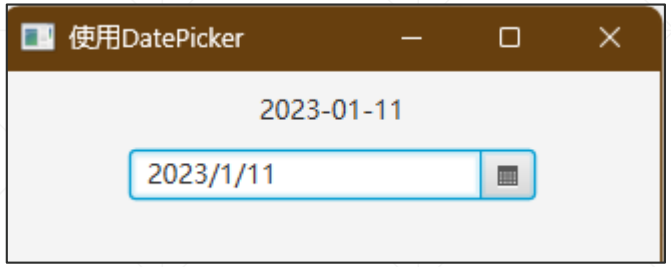



```
Label lblColor = new Label( text: "使用设置的颜色显示标签文本");
ColorPicker colorPicker = new ColorPicker();
colorPicker.setValue(Color.BLACK);
colorPicker.valueProperty().addListener(obj -> {
    lblColor.setTextFill(colorPicker.getValue());
});
```

UseColorPicker.java

使用DatePicker选择日期





UseDatePicker.java

```
DatePicker datePicker = new DatePicker();
Label lblDate = new Label( text: "选择一个日期");
datePicker.valueProperty().addListener(obj -> {
    lblDate.setText(datePicker.getValue().toString());
});
```